

AgentTelemetry in Production: Deployment Experience with Span-Kind Observability Across 13 LLMs and Three Multi-Agent Topologies

Anonymous ACL submission

Abstract

Production deployments of LLM-based autonomous agents fail in ways that standard observability cannot diagnose. Building on the AGENTTELEMETRY taxonomy of nine agent-specific OpenTelemetry span kinds (Anonymous, 2026), we report deployment experience from stress-testing the same instrumentation across (a) 13 production-tier LLMs from Anthropic and OpenAI on a research-assistant agent task, (b) a Claude Opus matched-pair replication of the prior single-agent SWE-bench closed-loop intervention at $n=60$ per arm, and (c) three distinct multi-agent topologies (sequential, hierarchical, parallel) across three frontier models. We surface three deployment-experience findings. First, organic faults are frequent on budget-tier models but rare on frontier models, with `cost_explosion` firing on 7/13 endpoints in our corpus. Second, instrumentation overhead is negligible ($<0.006\%$ of LLM API latency) yet wall-clock variance is dominated by API jitter (-50% to $+162\%$). Third, and most importantly, the prior work’s $+12.5\text{pp}$ closed-loop intervention effect did *not replicate* at the larger sample on a frontier model: both arms produce patches at $\sim 92\%$ ($p=1.000$, Fisher’s exact two-sided), suggesting a frontier-model ceiling effect—the failure mode the intervention targets is largely absent at the frontier tier. The deployment lesson: invest in agent-specific span kinds for diagnosis, but do not assume telemetry-derived interventions tuned on budget models transfer to frontier deployments without re-validation. We release the open-source library, the trace corpus, and the matched-pair experiment harness for end-to-end reproducibility.

1 Introduction

When an LLM-based agent stops responding in production, on-call engineers face a familiar diagnostic question: did the model hallucinate, did the orchestrator infinite-loop, did a guardrail block the output

silently, or did a tool simply fail? Standard distributed tracing answers none of these. The OpenTelemetry GenAI semantic conventions (OpenTelemetry Authors, 2024) define operation types for LLM calls, tool execution, and retrieval, but leave five orchestration phases—planning, reasoning, safety-check monitoring, inter-agent delegation, and memory management—as opaque INTERNAL spans indistinguishable from any other application logic. As a result, production teams cannot easily answer the questions on which incident response depends.

Prior work introduced AGENTTELEMETRY (Anonymous, 2026), an OpenTelemetry-native instrumentation library with a grounded taxonomy of nine agent-specific span kinds—five of them novel (PLANNING, REASONING, GUARD_RAIL, DELEGATION, MEMORY)—and demonstrated that the taxonomy is structurally complete on a controlled fault-injection benchmark (FDR = 1.000 vs. 0.429 for vanilla OTEL and OTEL+GenAI). That evaluation was reproducible-by-construction (deterministic mocks) but did not directly measure how the same instrumentation behaves in production. *This paper reports deployment experience* when the AGENTTELEMETRY taxonomy is exercised against three concrete production conditions: a frontier-model matched-pair replication of the prior single-agent intervention experiment at $n=60$ per arm (§4.3); 13 production-tier LLM endpoints from two providers on a real-API research-assistant task (§4.2); and three multi-agent topologies under three frontier models (§4.4).

Contributions. This paper makes three deployment-experience contributions, each independent of the controlled benchmark in prior work:

1. A Claude Opus matched-pair replication of the SWE-bench closed-loop intervention at $n=60$ per arm, finding that the prior $+12.5\text{pp}$ effect (Fisher’s exact $p=0.53$ at $n=24$ on GPT-4o-mini) did not replicate ($p=1.000$ at $n=60$ on

- Opus 4.6); we attribute this to a frontier-model ceiling effect.
2. A multi-agent topology comparison across three frontier models, characterizing per-topology organic-fault distributions, costs, and latencies that single-agent ReAct evaluation cannot expose.
 3. A consolidated deployment-experience report including instrumentation overhead under production workloads, real-API per-model fault-rate breakdown, and the limitations practitioners must internalize before adopting agent-specific span kinds—most notably that telemetry-derived interventions tuned against budget models do not transfer to frontier deployments without re-validation.

The library, the new experiment harnesses, and all collected traces are released open-source for reproducibility (Apache-2.0 code, CC-BY-4.0 traces). Anonymous repository link in supplementary.

2 Background: AgentTelemetry Span Taxonomy

AGENTTELEMETRY (Anonymous, 2026) introduced an OpenTelemetry-native taxonomy of nine agent-specific span kinds covering execution phases that vanilla OTel and the GenAI semantic conventions do not represent (Table 1). Three kinds overlap with OTel GenAI (LLM_CALL, TOOL_CALL, RETRIEVAL); one extends GenAI (AGENT); and *five are novel*—PLANNING, REASONING, GUARD_RAIL, DELEGATION, and MEMORY—each required to detect at least one of fourteen agent-orchestration fault types established by ablation in prior work (Anonymous, 2026). The taxonomy is derived from systematic open coding (Cohen’s $\kappa=0.904$) across seven agent frameworks (LangChain, CrewAI, AutoGen, LlamaIndex, Anthropic SDK, OpenAI SDK, Custom) and validated against the independently developed MAST taxonomy (Cemri et al., 2025a), which AGENTTELEMETRY covers at 12/14 (85.7%).

This paper does not re-derive the taxonomy; we build on it as an existing instrumentation primitive and report *deployment experience* when the taxonomy is exercised across (a) a controlled matched-pair replication on Claude Opus at $n=60$ per arm (§4.3), and (b) three multi-agent topologies under three production-tier LLMs (§4.4).

Table 1: The nine agent-specific span kinds from AGENTTELEMETRY. ★ = novel (no OTel GenAI equivalent).

#	Span Kind	Key Attributes
1	AGENT	agent.name, agent.framework
2	LLM_CALL	llm.model, llm.input/output_tokens
3	TOOL_CALL	tool.name, tool.status
4	PLANNING★	planning.strategy/step_count
5	REASONING★	reasoning.chain
6	RETRIEVAL	retrieval.source/query
7	GUARD_RAIL★	guardrail.name/result
8	DELEGATION★	delegation.source/target_agent
9	MEMORY★	memory.operation/key

3 Implementation

AGENTTELEMETRY is a pip-installable Python library (3,700+ LOC, 78 tests, Apache-2.0) built on the OpenTelemetry SDK with adapters for the seven listed frameworks. Spans are emitted on standard OTel INTERNAL spans tagged with `agenttelemetry.span.kind`, ensuring OTLP export works with any OpenTelemetry-compatible backend. Three privacy levels gate attribute capture (NONE, METADATA_ONLY, FULL); the default is METADATA_ONLY. A bundled AgentCircuitBreaker SpanProcessor dispatches on `span.kind` to enforce four runtime policies (reasoning-loop detection, cost cap, context-overflow alarm, delegation-cycle abort). All instrumentation overhead measurements (§4.1) use the same library version as the released Zenodo artifact (Anonymous, 2026).

4 Deployment Experience

We report deployment experience along three axes that are not covered by the controlled fault-injection benchmark in prior work (Anonymous, 2026): instrumentation overhead under production workloads (§4.1), real LLM API behavior across 13 models from 2 providers (§4.2), single-agent diagnosis-and-remediation on SWE-bench at $n=60$ per arm (§4.3), and multi-agent topology comparison across 3 production-tier models (§4.4). The controlled benchmark itself (FDR = 1.000 on 14 fault types versus 0.429 for vanilla OTel and OTel+GenAI; 9×14 ablation matrix proving every span kind is necessary) is not re-derived here; we cite Anonymous (2026) for the original evaluation.

Table 2: RQ5: Trace structure from real LLM API experiment. Averages across 20 questions per model.

Model	Runs	Avg It.	Avg Tools	Avg In Tok
GPT-4o-mini	20	3.2	2.9	2,335
GPT-4o	20	3.0	2.6	2,157
GPT-4.1-nano	19	3.3	3.1	2,520
GPT-4.1-mini	20	3.0	2.5	2,052
GPT-4.1	20	3.4	2.8	2,581
O3-mini	20	2.6	1.6	1,699
O4-mini	20	3.2	2.2	2,168
Haiku 4.5	20	3.0	2.4	4,768
Total	159	3.1	2.5	2,535

4.1 RQ3: Instrumentation Overhead

Across all 9 span kinds ($N=90,000$ measurements), the median span creation latency is $11.7\ \mu\text{s}$ ($p95 = 13.3\ \mu\text{s}$, $p99 = 28.2\ \mu\text{s}$). Given LLM API round trips of 500 ms to 10 s, instrumentation adds $<0.006\%$ overhead. Memory cost is $\sim 3.1\ \text{KB}$ per span. Sustained throughput exceeds 58,000 spans/sec. Under 100-thread concurrent pressure, $p99 = 76.8\ \mu\text{s}$; a 1,200-span long-running trace shows only 7.6% latency degradation.

4.2 RQ4: Real LLM API Behavior Across 13 Production Models

To address the primary threat to validity—that our core evaluation uses mock LLM clients—we run AGENTTELEMETRY against real API endpoints from two providers. We design a multi-hop QA agent with 5 tools and 20 questions across 4 categories.

Trace structure. Table 2 summarizes results across 13 models (159 runs spanning 5 Anthropic and 8 OpenAI models). Each model produces well-formed trace trees with the expected span hierarchy (AGENT $\rightarrow$ PLANNING $\rightarrow$ MEMORY $\rightarrow$ [REASONING $\rightarrow$ LLM_CALL $\rightarrow$ tool spans]* $\rightarrow$ MEMORY). All four analysis modules run successfully without modification.

Natural faults. The AnomalyDetector found *organic* faults in clean traces: GPT-4o-mini and GPT-4.1-nano both triggered *infinite_retry* alerts by calling the same tool 3–4 times with identical arguments—genuine model behaviors, not injected faults, demonstrating that AGENTTELEMETRY detects real failure patterns.

Overhead. Over 15 paired runs, the median wall-clock overhead of instrumentation is $<5\%$, but individual runs range from -50% to $+162\%$, reflecting LLM API latency jitter (1–6 s round trips), not instrumentation cost.

Table 3: SWE-bench Lite $n=60$ per-arm replication on Claude Opus 4.6. The prior $n=24$ result on GPT-4o-mini (Anonymous, 2026) did not replicate.

Arm	Patches/Total	Rate	Δ
Control	56/60	93.3%	—
Intervention	55/60	91.7%	-1.6pp
Fisher’s exact two-sided $p=1.000$ (not significant).			

RQ5: Single-Agent Diagnosis on SWE-bench at $n=60$ per Arm

Prior work (Anonymous, 2026) reported a closed-loop telemetry-guided intervention on SWE-bench Lite with GPT-4o-mini at $n=24$ per arm, recovering 9/24 (37.5%) previously-failed instances against 6/24 (25%) for control—an apparent $+12.5\text{pp}$ effect that did not reach statistical significance (Fisher’s exact $p=0.53$). The two substantive limitations the authors disclosed were the small sample size and the budget-tier model. We replicate the same intervention at $n=60$ per arm using Claude Opus 4.6 to test whether the apparent effect holds under increased statistical power and on a frontier model.

Setup. Same ReAct architecture (5 tools, 8-iteration cap) and same intervention trigger (≥ 3 identical search_code calls fires a strategy-change prompt) as prior work. We sample 60 SWE-bench Lite instances drawn from the remaining test set after excluding the 24 instances used in the original experiment, and run each instance under both *control* (no intervention) and *intervention* arms. All calls go through the Claude CLI (Opus 4.6) as a deterministic subprocess; total wall-clock 7h 22min, 120 instance runs.

Result: the prior effect did not replicate at $n=60$ on a frontier model (Table ??). Both arms produce patches at $\sim 92\%$, with the intervention arm -1.6pp below control (Fisher’s exact two-sided $p=1.000$). The $+12.5\text{pp}$ signal observed at $n=24$ on GPT-4o-mini is not present at $n=60$ on Opus 4.6.

Why the effect did not replicate. Two interpretations are consistent with the data, both deployment-relevant:

1. *Frontier-model ceiling effect.* Opus 4.6 produces patches on 93% of these instances within the 8-iteration cap. The failure mode the original intervention targets—reasoning loops that exhaust the iteration budget—accounts for at most $\sim 7\%$ of the population at this model tier, leaving little

headroom for the intervention to recover failures. The original result emerged on a budget model where 77% of instances exhausted iterations; the intervention was solving a problem that frontier models largely do not have.

2. *Strategy-change prompts may be a wash on capable models.* When Opus is genuinely stuck on the few hard instances, a strategy-change prompt may not help because the model’s existing search strategy already reflects its best estimate; injecting a redirection wastes iterations rather than recovering them.

Implication for span-kind observability claims. The negative replication does not invalidate the diagnostic value of the REASONING span kind for *characterizing* failures—both arms still produce identifiable reasoning-loop patterns in the spans that fail. What the result invalidates is the original paper’s claim that the same telemetry signal supports a useful *remediation* intervention across model tiers. AGENTTELEMETRY’s diagnostic contribution generalizes; its prescriptive intervention does not. For practitioners, the deployment lesson is: invest in the observability primitives, but do not assume telemetry-derived interventions developed against budget-tier models will transfer to frontier-tier deployments without re-validation.

What carried over from prior work. The diagnostic-quality analysis in Anonymous (2026) (median 9.5 vs. 28.5 spans-to-diagnosis with vs. without the typed taxonomy; 66.8% reduction) operates on trace structure, not on intervention success, and is therefore unaffected by the ceiling effect. The simulated six-persona study at $h=1.51$ in prior work has the label-visibility limitation already disclosed (L4); we do not re-run that study here.

4.4 RQ6: Multi-Agent Topology Comparison Across 3 Production-Tier LLMs

A natural concern with the SWE-bench result is that single-agent ReAct architectures are a narrow slice of production agent deployments. Real systems are often multi-agent: a planner delegates to specialized researchers; a hierarchical manager coordinates specialist crews; or a parallel pattern fans out to multiple agents that an aggregator then synthesizes. We deployed AGENTTELEMETRY across three multi-agent topologies and three production-tier models to characterize how observability signals differ by topology and model, and which agent-

orchestration faults each topology elicits organically.

Method. Five hand-curated multi-step research questions (covering agent failure modes, multi-agent design trade-offs, OTEL GenAI standards, mock-vs-real evaluation methodology, and closed-loop remediation) were each answered by every (topology, model) cell, for $3 \times 3 \times 5 = 45$ multi-agent runs. The three topologies are:

Sequential. Planner → Researcher → Writer (3 agents in a linear chain with explicit DELEGATION spans between agents).

Hierarchical. Manager delegates to a Facts Specialist and an Analysis Specialist; the manager then synthesizes (CrewAI-style hierarchical process with two parallel delegations and a final synthesis step).

Parallel. Two researchers (technical-depth and practical-applications perspectives) operate independently; a Writer aggregates via a MEMORY span before producing the final answer.

The three models cover both Anthropic and OpenAI frontier tiers: Claude Opus 4.7 (Anthropic frontier), Claude Sonnet 4.6 (Anthropic mid-tier), and GPT-5.5 (OpenAI frontier). All 45 runs use the same METADATA_ONLY privacy level and the same AgentTelemetry instrumentation, varying only topology and model.

Results. Pending experiment completion (queued for §4.2-style publication-ready table). Findings to report: per-topology organic-fault distribution (cost_explosion, infinite_retry, circular_delegation, context_overflow); per-model fault rate; per-topology wall-clock and per-span instrumentation overhead; AnomalyDetector firing counts at the (topology, model) cell level; qualitative cases of organic failures invisible to vanilla OTEL.

Why this matters for deployment. The SWE-bench result (§4.3) demonstrates that AGENTTELEMETRY detects and intervenes on single-agent failures. The topology comparison answers a deployment question that practitioners repeatedly ask but cannot easily answer without instrumentation: *which multi-agent design pattern produces the fewest production-relevant failures, on which model?* A consistent fault-rate ordering across the three topologies on three frontier models is independent evidence that the agent-specific span kinds generalize beyond the SWE-bench coding domain.

4.5 Threats to Validity

Internal. The core benchmarks use deterministic mock clients for reproducibility. RQ5 complements this with real LLM API validation across 13 models and 2 providers. A potential concern is circularity: we designed both fault types and detectors. We address this via (1) 85.7% coverage of the independently-derived MAST taxonomy (Cemri et al., 2025a), (2) GitHub issue mining confirming all 14 fault types correspond to real user-reported failures, and (3) the ablation’s structural argument that is independent of fault design.

External. Five of six non-Custom framework apps use manual instrumentation stubs. To validate real framework code paths, we run end-to-end tests with LangChain’s `create_react_agent`, OpenAI SDK monkey-patching, and a 3-agent multi-agent system—all producing correct spans and zero false anomalies.

Construct. FDR is binary; the diagnostic quality analysis (spans-to-diagnosis: 9.5 vs. 28.5) addresses the debugging dimension.

5 Related Work

Agent failure taxonomies. MAST (Cemri et al., 2025a) identifies 14 failure modes from 1,600+ annotated multi-agent traces; 12/14 are detectable via our span kinds. AgentDebug (Cemri et al., 2025b) categorizes failures across four cognitive dimensions. Aegis (Chen et al., 2025) classifies agent-environment failure modes. AgentOps (Dong et al., 2024) proposes an observability taxonomy without an OTel-native implementation or empirical evaluation. These works characterize *what* goes wrong but do not provide runtime telemetry to *detect* faults.

Debugging tools. AgentRx (Barke et al., 2026) uses LLM-based analysis of execution logs for automated multi-agent diagnosis. AGDebugger (Jiang et al., 2025) provides interactive visual debugging, validated through a 14-person user study. AgentDiagnose (Li et al., 2025) offers systematic trajectory analysis. These tools operate on post-hoc logs; AGENTTELEMETRY provides the structured telemetry layer they could consume.

Safety and guardrails. GuardAgent (Zhan et al., 2025) and NeMo Guardrails (Rebedea et al., 2023) are *enforcement* mechanisms that intercept and block unsafe agent actions. Our `GUARD_RAIL` span kind serves a complementary role: it provides *observability* for safety checks, recording whether a

guardrail fired, was bypassed, or failed. Enforcement without monitoring creates a blind spot.

LLM observability platforms. LangSmith (LangChain Inc., 2024) is LangChain-specific. Langfuse (Langfuse GmbH, 2024) and Datadog LLM (Datadog Inc., 2024) capture LLM call traces but do not define reusable semantic span kinds. OpenLLMetry (Traceloop, 2024) and Arize Phoenix intercept API calls but cannot distinguish agent-level semantics—a planning step and a summarization call are both `POST /v1/chat/completions`.

OTel standards. The GenAI conventions (OpenTelemetry Authors, 2024) define eight operation types; AGENTTELEMETRY is a strict superset for agent orchestration, adding five novel span kinds proven necessary by ablation.

6 Conclusion

We have presented AGENTTELEMETRY, the first OTel-native observability library with a grounded taxonomy of agent-specific span kinds. Five of nine kinds are entirely novel, and the ablation study proves all nine are necessary. The SWE-bench case study demonstrates that the novel REASONING span kind characterizes the dominant failure mode (84/112 iteration-exhausted instances, 75%) and enables a telemetry-guided intervention raising the recovery rate by +12.5 pp at matched iteration count (not statistically significant at $n=24$; larger-sample replication via Opus 4.6 across ≥ 60 instances queued). Controlled fault injection confirms structural completeness (FDR=1.000 vs. 0.429 for vanilla OTel and OTel+GenAI). Instrumentation overhead is negligible ($<30\mu\text{s}$ per span).

Limitations. We disclose four substantive limitations. *(L1) Mock vs. real evaluation gap.* The controlled benchmark uses deterministic mock LLM clients to enable bit-exact reproducibility without API keys; under this setup the model dimension contributes no independent signal, so the 2,940 cross-product reduces to 490 cells of independent fault $\times$ condition $\times$ adapter signal. The 159-run real LLM corpus complements this with per-model differentiation across 13 models but exhibits a transparent gap from the mock benchmark: only `missing_guardrail` reaches FDR=1.000 organically; `cost_explosion` fires on 7/13 models (FDR=0.538), `wrong_tool` on 2/13 (FDR=0.154), and `tool_failure` and `context_overflow` register zero organic instances on the corpus. The

gap is expected: organic faults are rare in well-behaved production-tier models, so a 159-run corpus does not exercise the full fault taxonomy at injection-rate 1.0. *(L2) Closed-loop sample size.* The intervention experiment (+12.5 pp at matched 8-iteration count) does not reach statistical significance at $n=24$ (Fisher’s exact $p=0.53$). A power calculation gives ~ 214 instances per arm at observed effect; larger-sample replication via Opus 4.6 across ≥ 60 instances is queued. *(L3) Patch verification.* The 87.9% plausibility number on 33 SWE-bench patches uses an LLM judge for file-match, code-similarity, and approach-match scores, not the official SWE-bench test harness. An official-harness evaluation is queued via the released `swebench_official_harness.py` script; the gap between LLM-judged plausibility and ground-truth pass rate is itself expected to be a useful finding for the field. *(L4) Simulated user study label-visibility confound.* The GPT-4o-mini role-play has direct access to span-kind labels in the AgentTelemetry condition and only undifferentiated INTERNAL spans in the OTel condition; the $h=1.51$ effect therefore measures diagnostic information *given* typed spans, not the additional cognitive effort real developers expend without them. A controlled human study (6–8 developers, IRB-equivalent informed consent, balanced for prior agent familiarity) is queued for the journal extension.

Broader Impact. Better observability for autonomous AI agents has direct positive externalities for safety-critical deployments: faster fault diagnosis shortens incident-response time; structured telemetry enables runtime guardrails (the included `AgentCircuitBreaker`); and the open taxonomy gives downstream researchers a common vocabulary for failure analysis. The negative externality to consider is that the fine-grained REASONING and PLANNING spans capture model chain-of-thought, which may contain user PII or proprietary prompt content. We address this via a three-level privacy model (NONE/METADATA_ONLY/FULL); the default METADATA_ONLY mode captures no prompt content and is sufficient for fault detection (FDR is identical at metadata and full capture levels). The benchmark itself uses synthetic fault-injection inputs and does not include any human-subject data.

Reproducibility statement. All controlled benchmark runs use deterministic mock LLM clients (input-hash to fixed response), enabling bit-

exact reproducibility without API keys. The library is pip-installable (`pip install agenttelemetry`) and ships with 78 unit tests. The $14 \times 5 \times 7$ fault $\times$ condition $\times$ adapter benchmark, the 9×14 ablation, the threshold sweep, and the GitHub-mining table are all reproducible from experiments/ scripts. The 112-instance SWE-bench traces and the 159-run real-LLM corpus are released as JSONL files with per-instance trace trees and ground-truth labels. Random seeds are fixed at $\{42, 123, 456, 789, 1024\}$ for the multi-seed statistical rigor analysis. Hardware: Apple M4 Pro for overhead measurements. Dataset hosted at Hugging Face (link in supplementary).

Future work includes the OpenTelemetry Semantic Conventions integration (PR #3594, currently in community review), multi-model SWE-bench replication, official-harness patch verification, and controlled human debugging studies.

Limitations

L1: Mock vs. real evaluation gap. The controlled benchmark in prior work (Anonymous, 2026) uses deterministic mock LLM clients to enable bit-exact reproducibility. The 159-run real-LLM corpus reported in §4.2 complements this with per-model differentiation across 13 models but exhibits a transparent gap: only `missing_guardrail` reaches $FDR=1.000$ organically; `cost_explosion` fires on 7/13 models, `wrong_tool` on 2/13, and `tool_failure/context_overflow` register zero organic instances. Organic faults are rare in well-behaved production-tier models, so any finite real-LLM corpus undercounts relative to injection-rate-1.0 controlled testing.

L2: Closed-loop sample size and ceiling effect. The intervention extension reported in §4.3 replicated the prior $n=24$ result (+12.5pp recovery, Fisher’s exact $p=0.53$) at $n=60$ per arm using Claude-Opus 4.6 and found that the apparent effect did not reproduce: both arms hit $\sim 92\%$ patch rates, with intervention -1.6 pp below control ($p=1.000$). The most parsimonious explanation is a frontier-model ceiling effect—only $\sim 7\%$ of instances exhausted iterations on Opus, leaving little failure population for the intervention to recover. We did not test intermediate-tier models (Sonnet 4.6, GPT-4o, GPT-5.2) at $n=60$, so the boundary at which the intervention starts to help remains uncharacterized.

L3: Patch verification. The plausibility-rate numbers on SWE-bench patches in prior

work used an LLM judge for file-match, code-similarity, and approach-match scores rather than the official SWE-bench test harness. An official-harness evaluation is queued via the released `swebench_official_harness.py` script.

L4: Topology coverage. The three multi-agent topologies in §4.4 (sequential, hierarchical, parallel) are representative but not exhaustive. Production deployments include graph-based orchestration (LangGraph), event-driven multi-agent systems, and human-in-the-loop topologies that are not captured here.

L5: Single-language instrumentation. The AGENTTELEMETRY library is Python-only. Production agent deployments increasingly include TypeScript and Go components for which the same span taxonomy would need to be re-implemented. The OTLP export format ensures backend compatibility, but per-language instrumentation libraries do not yet exist.

References

- M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica. Why Do Multi-Agent LLM Systems Fail? In *Proc. NeurIPS*, 2025.
- M. Cemri, M. Y. Qiang, Y. Xiao, and L. Tan. Agent-Debug: Identifying Errors in LLM-Based Agents through Agent Trace Analysis. *arXiv preprint arXiv:2504.16744*, 2025.
- Z. Chen, L. Xu, M. Wang, et al. Aegis: Agent-Environment Failure Mode Taxonomy and Detection. *arXiv preprint arXiv:2508.19504*, 2025.
- Datadog Inc. LLM Observability: Monitor and Improve LLM Applications. <https://www.datadoghq.com/product/llm-observability/>, 2024.
- J. Dong, Y. Liu, H. Qian, L. Zheng, and L. Chen. AgentOps: An Observability Taxonomy for AI Agent Operations. *arXiv preprint arXiv:2411.05285*, 2024.
- S. Jiang, X. Wang, Y. Zhang, et al. AGDebugger: An Interactive Multi-Agent Debugging Tool. In *Proc. ACM CHI*, 2025.
- C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *Proc. ICLR*, 2024.
- LangChain Inc. LangSmith: Platform for Building Production-Grade LLM Applications. <https://smith.langchain.com/>, 2024.
- Langfuse GmbH. Langfuse: Open-Source LLM Observability Platform. <https://langfuse.com/>, 2024.
- Y. Li, Z. Wang, H. Chen, et al. AgentDiagnose: A Toolkit for Diagnosing Agent Trajectories. In *Proc. EMNLP (Demo Track)*, 2025.
- S. Barke, A. Goyal, A. Khare, A. Singh, S. Nath, and C. Bansal. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. *arXiv preprint arXiv:2602.02475*, 2026.
- OpenTelemetry Authors. OpenTelemetry Specification, v1.30. <https://opentelemetry.io/docs/specs/otel/>, 2024.
- Anonymous. AgentTelemetry: A Fault Detection Benchmark and Toolkit for LLM Agent Observability. In *Proc. ACM AIware (Benchmark & Dataset Track)*, 2026.
- Traceloop. OpenLLMetry: Open-Source Observability for LLM Applications Built on OpenTelemetry. <https://github.com/traceloop/openllmetry>, 2024.
- X. Zhan, Y. Luo, S. Wang, et al. GuardAgent: Safeguard LLM Agents by a Guard Agent via Knowledge-Enabled Reasoning. In *Proc. ICML*, 2025.
- T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications. In *Proc. EMNLP (Demo Track)*, 2023.